

# LLM-Based Analysis of User Comments for HarmonyOS Applications

AGCOMMENTSPY — From Raw Reviews to Actionable Insights

Lin Wenqi   Hu Haozhan   Wang Zhaohe

CS326 Project

May 31, 2026

# Outline

- 1 Introduction
- 2 Contributions
- 3 System Architecture
  - Comment Scraping
  - LLM Classification
  - Topic Clustering
  - Web Dashboard
- 4 Results
- 5 Evaluation & Discussion
- 6 Conclusion & Future Work

# Motivation

## HarmonyOS Ecosystem

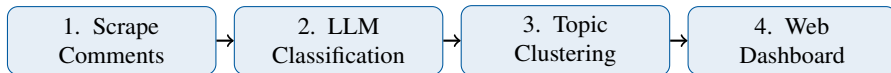
- Huawei's distributed OS — smartphones, tablets, wearables, IoT
- AppGallery hosts **500,000+** native HarmonyOS apps
- User comments are a critical feedback channel for developers

## Unique Challenges of HarmonyOS

- **Closed ecosystem:** HDC protocol, limited tooling (no adb)
- **Heterogeneous devices:** phones, foldables, watches, in-vehicle
- **Rapid API evolution:** compatibility issues surface in reviews

Existing tools target Google Play / App Store — HarmonyOS needs its own pipeline.

# Our Approach at a Glance



## Key Insight

Use UI automation to collect reviews + LLM to classify & cluster → transform **unstructured reviews** into **actionable developer insights**.

# Contributions

- ① **UI-automation crawler** — reliably scrapes AppGallery comments via `hmdriver2` + HDC
- ② **LLM classification pipeline** — multi-dimensional sentiment analysis + actionable suggestions
- ③ **Two-stage clustering** — intra-bucket grouping + cross-batch refinement
- ④ **Interactive web dashboard** — aggregated results in intuitive visual format
- ⑤ **Curated dataset** — **10,000+** comments across 8 categories on HarmonyOS

# System Architecture — Overview

**Comment Scraping → LLM Classification → Topic Clustering → Web Dashboard**

HDC + hmdriver2 | LLM API (JSON mode) | 2-stage LLM cluster | Python HTTP + Vanilla JS

## Technology Stack

- hmdriver2 + HDC — device automation
- lxml + XPath — UI parsing
- OpenAI-compatible API — LLM calls
- Python 3.13 — all backend logic
- Vanilla JS + SVG — frontend

## Data Flow

- Raw comments → JSON files
- Batch LLM classification (10/batch)
- Classification → clustering
- REST API → dashboard

# Stage 1: Comment Scraping

## How it works

- Launch AppGallery via deep-link URL
- **Configurable swipes** to load reviews (max 500)
- Parse UI hierarchy (JSON → XML, XPath)
- Extract: username, rating, text, date, device
- **Auto-expand** truncated long comments
- Save per-app JSON files

## Scraping Loop (simplified)

```
1 def scrape_comments(driver, max_swipes=50):
2     all_comments = []
3     for i in range(max_swipes):
4         driver.swipe(...)
5         layout = driver.dump_hierarchy()
6         xml = json2xml(layout)
7         for node in xml.xpath(COMMENT_XPATH):
8             c = parse_comment(node)
9             all_comments.append(c)
10    return all_comments
```

# Stage 2: LLM-Based Classification

## Goal

Raw app reviews → structured developer insights

## Classification

- **Sentiment:** Positive, Negative, Neutral
- **7 dimensions:**
  - Functional Experience
  - System Compatibility
  - Stability & Performance
  - UI & Interaction
  - Operations & Policies
  - Appreciation
  - Content Ecosystem

## Structured Output

- Pain point + actionable suggestion
- Comment ID for traceability

## Reliability

- JSON schema, 10 comments/batch
- Auto-repair malformed responses

## Example

### Raw review

“The HarmonyOS version crashes when opening videos, and landscape mode is poorly adapted.”

### LLM classification output

- **Sentiment:** Negative
- **Dimension:** Stability & Performance
- **Pain point:** Video playback crashes
- **Suggestion:** Check crash logs; improve landscape adaptation



# Stage 3: Two-Stage Topic Clustering

## Initial attempt: word-embedding clustering

- Sentence-BERT + KMeans/DBSCAN
- **Failed:** same root cause, different wording → far apart in vector space → split into different clusters

## Our approach: LLM-as-clusterer

- **Stage 1:** partition by (dimension, sentiment); LLM clusters items within each bucket in batches of 18
- **Stage 2:** merges overlapping clusters across batches and standardizes topic names

### Example

**cluster\_name:** 好友信息功能缺失

**canonical\_issue:** 鸿蒙版微信缺少好友来源、共同群聊、添加好友时间等常用功能

**size:** 2

**#264** 华为畅享90 Pro Max

“好友来源、好友共同群聊、添加好友时间等都没有，求更新”

**#272** HUAWEI Pura 70 Pro+

“连朋友资料都没，版本比别人低...”

# Stage 4: Web Dashboard

## Four Tabs

### ① Overview

- ▶ Total comments, classified count
- ▶ Donut chart (sentiment)
- ▶ Bar chart (dimensions)

### ② Comments

- ▶ Paginated list + full-text search
- ▶ Username, rating, date, device

### ① Analysis

- ▶ Dimension-filtered pain points
- ▶ LLM-generated **actionable suggestions**

### ② Clusters

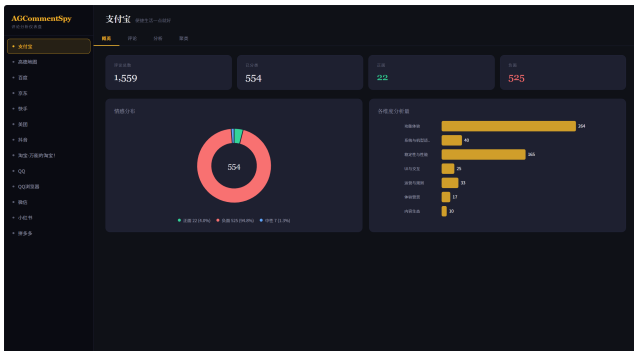
- ▶ Topics by dimension  $\times$  sentiment
- ▶ Expandable sample members

## Tech Stack

Vanilla JS (no framework) + inline SVG charts

Python HTTP server + REST API endpoints

# Results — Dashboard Overview



- **Dataset: 10,000+** comments across **45+** apps in 8 categories
- **Sentiment distribution** visualized as an SVG donut chart
- **Dimension breakdown** shown as a horizontal bar chart
- One-click access to total/classified counts

Figure: Overview — aggregate statistics and charts



# Evaluation & Discussion

## Classification

- 7 dimensions balance **granularity** with **actionability**
- Every negative issue → concrete Actionable\_Suggestion

## Clustering

- Word-embedding clustering separated same-root-cause items (lexical, not semantic)
- LLM-as-clusterer reasons about root causes, merging despite vocabulary variance
- Two-stage design: focused sub-problems + lightweight reconciliation

# Conclusion

## What We Built

AGCOMMENTSPY — end-to-end pipeline that transforms **unstructured AppGallery reviews** into **actionable developer insights** via UI automation + LLM-based NLP.

## Key Takeaways

- UI automation compensates for the **lack of public APIs** on HarmonyOS
- LLMs enable **zero-shot** fine-grained classification of user reviews
- Two-stage clustering turns **scattered comments** into **coherent topics**
- Lightweight web dashboard makes results **accessible to developers**

# Future Work

- ① **Automated labeling** — human-annotated gold standard for evaluation
- ② **LLM comparison** — benchmark multiple providers on cost-quality
- ③ **Incremental scraping** — only scrape new comments since last run
- ④ **Developer workflow integration** — auto-file findings to GitHub Issues / Jira

# Thank You!

Questions & Discussion

AGCOMMENTSPY — CS326 Project